# Neural Networks and Deep Learning
# Assignment 2

**Submitted by: Lokesh Kumar Sihag**

**Roll No: 2301201204**

## 1. What is epoch, batch size, and Stratified, and please explain them.

An **epoch** in machine learning refers to one complete iteration over the entire training dataset. During training, the model learns by repeatedly processing the data, and each full pass through all the training samples is counted as one epoch. Multiple epochs are usually required for the model to learn meaningful patterns from the data. However, while increasing the number of epochs can improve learning, too many epochs may lead to overfitting, where the model performs well on training data but poorly on unseen data.

The **batch size** is the number of training samples that are processed together in a single step before updating the model's parameters. Instead of feeding the entire dataset at once, the data is divided into smaller subsets called batches. This approach makes training more efficient and manageable, especially for large datasets. A smaller batch size can lead to more frequent updates and potentially better generalization, while a larger batch size can speed up computation but requires more memory.

The term **stratified** refers to a method used in data splitting where the distribution of classes is preserved across different subsets of the dataset, such as training and testing sets. In a stratified split, each subset maintains the same proportion of class labels as the original dataset. This is particularly important when dealing with imbalanced data, as it ensures that all classes are fairly represented, leading to more reliable training and evaluation of the model.

## 2 Explain the different types of activation functions.

Activation functions are an essential component of neural networks, as they determine how the weighted sum of inputs is transformed into an output signal for the next layer. They introduce non-linearity into the model, allowing neural networks to learn complex patterns rather than just linear relationships.

One of the simplest activation functions is the **linear activation function**, where the output is directly proportional to the input. Although it is easy to compute, it is rarely used in hidden layers because it does not introduce non- linearity, making the model behave like a simple linear regression.

The **sigmoid activation function** maps input values into a range between 0 and 1, making it especially useful for binary classification problems. It produces an S-shaped curve, which helps in interpreting outputs as probabilities. However, it suffers from the vanishing gradient problem, where gradients become very small and slow down learning in deep networks.

The **tanh (hyperbolic tangent) function** is similar to sigmoid but maps input values between -1 and 1. Because it is centered around zero, it often performs better than sigmoid in hidden layers. Despite this advantage, it still faces the vanishing gradient issue in deep networks.

The **ReLU (Rectified Linear Unit)** is one of the most widely used activation functions. It outputs zero for negative input values and the input itself for positive values. ReLU is computationally efficient and helps reduce the vanishing gradient problem, which makes it suitable for deep neural networks. However, it can suffer from the "dying ReLU" problem, where neurons may stop updating if they consistently output zero.

To address this limitation, variations such as **Leaky ReLU** are used. Leaky ReLU allows a small, non-zero output for negative input values, which helps keep neurons active and improves learning.

Another commonly used function is the **softmax activation function**, which is typically applied in the output layer for multi-class classification problems. It converts raw output values into probabilities that sum up to one, making it easier to interpret the model's predictions.

## 3. How we handle structured and unstructured data in ANNs and CNNs.

Artificial Neural Networks (ANNs) and Convolutional Neural Networks (CNNs) handle structured and unstructured data differently based on the nature and organization of the data.

**Structured data** refers to data that is organized in a clear, tabular format, such as rows and columns (for example, spreadsheets or databases). In ANNs, structured data is typically used as input in the form of feature vectors, where each column represents a feature and each row represents an observation. Before feeding the data into the network, it is usually preprocessed through normalization, encoding

of categorical variables, and handling of missing values. ANNs are well-suited for this type of data because they can learn relationships between features through fully connected layers. CNNs, on the other hand, are not commonly used for structured data because they are designed to capture spatial patterns, which structured tabular data generally does not have.

**Unstructured data** includes data that does not follow a fixed format, such as images, audio, and text. CNNs are particularly effective for handling unstructured data like images because they use convolutional layers to automatically detect spatial features such as edges, textures, and shapes. These layers preserve the spatial relationships between pixels, making CNNs highly efficient for tasks like image classification and object detection. For text and sequential data, preprocessing steps such as tokenization and embedding are applied before feeding the data into neural networks. While traditional ANNs can also process unstructured data after it is converted into numerical form, they do not perform as well as CNNs in capturing spatial or hierarchical patterns.

## 4. What is a layer, and define ANN and CNN layers?

In neural networks, a **layer** is a fundamental building block that consists of a group of neurons working together to process input data and produce an output. Each layer receives input, applies a mathematical transformation (such as a weighted sum followed by an activation function), and passes the result to the next layer. By stacking multiple layers, a neural network can learn increasingly complex patterns from the data.

In an **Artificial Neural Network (ANN)**, layers are typically organized into three main types: the input layer, hidden layers, and the output layer. The input layer receives the raw data in the form of feature vectors. The hidden layers, which lie between the input and output layers, perform most of the computation by applying weights, biases, and activation functions to learn patterns in the data. These layers are usually fully connected, meaning every neuron in one layer is connected to every neuron in the next layer. The output layer produces the final result, such as a predicted value or class label. ANN layers are best suited for structured data where relationships between features are learned through these dense connections.

In a **Convolutional Neural Network (CNN)**, layers are designed specifically to process data with spatial structure, such as images. In addition to input and output layers, CNNs include specialized layers like convolutional layers, pooling layers, and fully connected layers. Convolutional layers apply filters (kernels) that slide over the input to extract important features such as edges and textures. Pooling layers reduce the dimensionality of the data by summarizing regions, which helps in reducing computation and controlling overfitting. After several convolutional and pooling operations, the data is flattened and passed to fully connected layers,

similar to those in ANNs, for final classification or prediction. Thus, CNN layers are structured to automatically learn spatial hierarchies of features from unstructured data like images.

## 5. What are the concepts used in ANN and CNN?

In **Artificial Neural Networks (ANNs)**, one of the core concepts is the **neuron (or perceptron)**, which takes input values, applies weights and a bias, and passes the result through an activation function to produce an output. Another key concept is **weights and biases**, which are the parameters the model learns during training to minimize error. The **activation function** introduces non- linearity, allowing the network to learn complex relationships. **Forward propagation** is the process by which input data moves through the network layer by layer to produce an output, while **backpropagation** is the method used to update weights by calculating gradients of the error. ANNs also rely on a **loss function**, which measures how far the predicted output is from the actual output, and an **optimizer**, which adjusts the weights to reduce this loss. Additionally, concepts like **epochs**, **batch size**, and **learning rate** control how the model is trained.

In **Convolutional Neural Networks (CNNs)**, many ANN concepts are still used, but there are additional specialized ideas for handling spatial data. The most important concept is the **convolution operation**, where filters (kernels) slide over the input to extract features such as edges and patterns. **Feature maps** are the outputs produced by these filters, representing detected features. **Pooling** (such as max pooling) is another concept used to reduce the size of feature maps while retaining important information, which helps in reducing computation and overfitting. CNNs also use **stride and padding** to control how filters move across the input and how the output size is maintained. After convolution and pooling, the data is **flattened** and passed into fully connected layers for final prediction. Like ANNs, CNNs also use activation functions, loss functions, and backpropagation for learning.